

03 — Evolution Engine

Objetivo de este documento: documentar, contra el código real, cómo aprende Puglit. Cómo una victoria en el torneo se convierte en XP, niveles y una lección embebida; cómo esa lección se almacena, se recupera por relevancia, envejece y se consolida; y cómo el sistema se defiende del *drift* y del *poisoning* de su propia memoria. Toda fórmula, esquema y umbral acá está verificado contra `web/lib/progression.ts` , `web/lib/skill-evolution.ts` , `web/lib/swarm-fitness.ts` , `web/lib/swarm-metrics.ts` , `web/lib/embed.ts` y `web/sql/genetic.sql` .

1. Learning Model

Puglit no entrena pesos. Los 75 agentes (3 teams × 25 roles, `web/lib/roster.ts`) son modelos Ollama **congelados**. Lo que evoluciona es **estado textual y estructurado** alrededor de esos modelos. Hay tres tipos de estado aprendido, y cada uno tiene un mecanismo distinto, un lugar de persistencia en Postgres y una compuerta de calidad propia:

Estado aprendido	Qué es	Dónde vive	Cómo cambia	Compuerta
Progresión RPG	XP, nivel, stats, reputación de calidad por agente	<code>puglit_agents</code>	<code>awardRound()</code> tras cada round del torneo	victoria/participación
Diary (genes)	Lecciones embebidas (qué funcionó / qué mejorar)	<code>puglit_agent_diary</code>	una entrada por área-puntuada, por agente relevante	<code>quality≥45</code> + <code>outcome≠failure</code> al recuperar
Skills (SkillOpt)	El <i>skill doc</i> de cada área, editable y versionado	<code>puglit_skills</code> / <code>puglit_skill_rejects</code>	<code>evolveSkill()</code> offline	<i>held-out validation gate</i> (estrictamente mejor por margen)

La idea central es **separar mecanismo de evidencia**. La progresión RPG (XP/niveles) es el mecanismo lúdico que hace observable la dinámica genética en el campus 2.5D, pero **no es la métrica que valida al sistema**: eso lo hace `swarm-metrics.ts` (build success, smoke pass, acuerdo entre jueces, ablación swarm-vs-single). El aprendizaje *que mueve la calidad* son las lecciones del diary (inyectadas como few-shot) y los skill docs evolucionados (inyectados como system prompt). Ambos se ganan por victorias, pero **solo se aplican si pasan una compuerta**.

El bucle completo:



2. XP Calculation (Participation · Victory · Category Bonus)

El Grand Jury (Stakeholder + 4 especialistas) puntúa el entregable de **cada team por área**, de 1 a 100 — nunca a los 75 agentes uno por uno (sería carísimo en tokens). `awardRound()` (`progression.ts:72`) distribuye ese puntaje a los agentes individuales con esta fórmula (`progression.ts:9`, `:88-92`):

```
XP_gained = areaScore(area_del_agente) × victoryFactor(team) × participation(role, round)
```

Verificado contra el código:

- **areaScore** — el puntaje 0-100 del área a la que pertenece el rol (`ROLE_AREA`, `progression.ts:28`). Las cuatro áreas son `data | dev | design | business`. La **queen** es el caso especial: cobra el **mejor** área de su team, porque es dueña del entregable completo (`progression.ts:88`):

```
ts const area = a.queen ? "data" : (ROLE_AREA[a.role] || "business") const score = a.queen ? Math.max(...AREAS.map((ar) => teamScores[ar] ?? 0)) : (teamScores[area] ?? 0)
```

- **victoryFactor** — `1.5` para el team ganador, `0.8` para los perdedores (`progression.ts:90`). Los perdedores ganan menos, **pero igual aprenden** (cobran XP y escriben lección). Es selección blanda, no eliminación.

```
ts const victory = a.team === winner ? 1.5 : 0.8
```

- **participation** — cuánto trabajó ese rol *en ese tipo de round* (`PARTICIPATION`, `progression.ts:42`). El round `diverge` (iteración 1) es diseño de blueprint: los architects y researchers hacen el trabajo (`1.0 - 0.8`), los builders apenas participan (`0.2 - 0.15`). El default si no hay registro es `0.2`

(`participationFor` , `progression.ts:52`). La **queen** siempre participa al `1.0` (`progression.ts:91`).

El resultado se redondea: `const gained = Math.round(score * victory * part) (:92)`, y si `gained <= 0` el agente se saltea (`:93`).

Sobre el "Category Bonus". No hay un multiplicador de categoría explícito por encima de la fórmula; el "bonus de categoría" está **codificado en el mapeo `ROLE_AREA`**: cada rol cobra el puntaje de *su* área, así que un buen score de diseño beneficia a los roles de `design` y no a los de `data`. El único *bonus* estructural sobre la fórmula base es el de la queen (toma `max` de áreas) y el factor victoria. Hago la aclaración para no inventar un campo que el código no tiene.

Reputación de calidad. En paralelo al XP, cada agente acumula una reputación: el score de área 0-100 se normaliza a 0-10 y se suma (`progression.ts:105`):

```
const qsum = a.quality_sum + score / 10    // 0-100 → contribución 0-10
const qn    = a.quality_n + 1
```

Esto persiste en `puglit_agents.quality_sum / quality_n` (la "opinión de la Queen sobre el agente a lo largo de muchos proyectos", `genetic.sql:34`). Esa misma normalización `/10` es la que se guarda como `quality` de la lección en el diary — y es la que después alimenta la compuerta anti-poisoning del § 8.

3. Leveling System (Fórmulas · Umbrales)

La curva de niveles es RPG clásica, niveles 1–1000 (`progression.ts:55`):

```
export const xpToReach = (level: number) => Math.floor(100 * Math.pow(level, 1.8))
```

Es decir: **XP-para-alcanzar(L) = $\lfloor 100 \cdot L^{1.8} \rfloor$** . Algunos umbrales reales que produce esa fórmula:

Nivel L	XP acumulado para alcanzarlo
1	100
2	348
3	725
5	1.810
10	6.309
20	21.984
50	120.940
100	398.107

El exponente `1.8` (sub-cuadrático) hace que subir de nivel sea cada vez más caro pero nunca imposible: a más nivel, más victorias y más calidad hace falta. La conversión inversa (`levelForXp` , `progression.ts:56`) arranca de una estimación cerrada y ajusta hacia arriba:

```
export function levelForXp(xp: number): number {
  if (xp <= 0) return 1
  let L = Math.max(1, Math.floor(Math.pow(xp / 100, 1 / 1.8)))
  while (xpToReach(L + 1) <= xp) L++
  return L
}
```

Subida de stat por nivel. Cada nivel ganado suma **+1 a la stat de especialidad** del agente (la más alta de su set `{creativity, rigor, security, speed, depth}`), por cada nivel ganado en ese round (`progression.ts:100-103`):

```
if (levelsGained > 0) {
  const spec = Object.keys(stats).sort((x, y) => (stats[y]||0) - (stats[x]||0))[0]
  if (spec) stats[spec] = (stats[spec]||0) + levelsGained
  leveledUp.push({ id: a.id, level: newLevel })
}
```

Esto cierra el bucle genético: las stats RPG **derivan los parámetros de Ollama** del agente (`temperature` /determinismo, `genetic.sql:27-29`), así que un agente que gana consistentemente en su especialidad se vuelve gradualmente más "afilado" en los parámetros con que llama al modelo. El nuevo estado se persiste de una sola escritura por agente (`progression.ts:108-112`), que además incrementa `projects` y `wins`.

4. Knowledge Extraction (cómo una victoria genera aprendizaje)

Cada área puntuada genera **una lección** para cada agente relevante — ganadores **y** perdedores. La extracción es determinística (no hay una segunda llamada al LLM para "reflexionar"): se compone del puntaje del área y del `feedback` que el jurado ya emitió por área (`progression.ts:114-118`):

```
const fb = feedback[a.team]?.[area]
const entry = a.team === winner
  ? `Ganamos el round (${AREA_ES[area]} ${score}/100). ${fb || "Mantener este enfoque."}`
  : `Perdimos (${AREA_ES[area]} ${score}/100). A mejorar: ${fb || "subir fidelidad y completitud"}`
```

- El **ganador** registra *qué funcionó* (`kind = "win"`).
- El **perdedor** registra *la crítica a mejorar* (`kind = "critique"`).

Esto es deliberado: el sistema aprende tanto de la victoria como de la derrota, y la derrota se guarda en forma accionable ("a mejorar: ..."), no como castigo.

Acto seguido, la lección se **embebe** para poder recuperarla luego por **relevancia semántica** y no solo por recencia (`progression.ts:119-125`). El embedding es el "gen" de esa lección:

```

if (!embCache.has(entry)) embCache.set(entry, await embed(entry).catch(() => null))
const emb = embCache.get(entry) || null
await query(
  `INSERT INTO puglit_agent_diary (agent_id, job_id, kind, entry, quality, embedding) VALUES ($1, $2, $3, $4, $5, $6)`
  [a.id, jobId, a.team === winner ? "win" : "critique", entry, score / 10, emb ? JSON.stringify(emb) : null]
)

```

Detalle de eficiencia real: muchos agentes de un mismo team comparten la **misma** lección textual, así que `embCache` (un `Map`, `progression.ts:82`) embebe cada string único **una sola vez** por round, en vez de 25 veces. Si el embedding falla, se guarda `null` y el sistema cae a recencia (graceful degradation, ver § 6).

5. Embedding Storage (Esquema · Índices)

El diary es la memoria de largo plazo. Esquema real (`genetic.sql:41-53`):

```

CREATE TABLE IF NOT EXISTS puglit_agent_diary (
  id          BIGSERIAL PRIMARY KEY,
  agent_id    VARCHAR(64) NOT NULL REFERENCES puglit_agents(id),
  job_id      VARCHAR(32),
  kind        VARCHAR(16) NOT NULL DEFAULT 'lesson', -- lesson | win | critique | note
  entry       TEXT NOT NULL,
  quality     DOUBLE PRECISION, -- score de la Queen para esta contribución (0..10)
  embedding   JSONB, -- "gen" vector (nomic-embed) para retrieval por relevancia
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_diary_agent ON puglit_agent_diary(agent_id, created_at DESC);
ALTER TABLE puglit_agent_diary ADD COLUMN IF NOT EXISTS embedding JSONB;
-- migraciones idempotentes posteriores:
ALTER TABLE puglit_agent_diary ADD COLUMN IF NOT EXISTS outcome VARCHAR(12) DEFAULT 'unknown';
ALTER TABLE puglit_agent_diary ADD COLUMN IF NOT EXISTS scope VARCHAR(16) DEFAULT 'team';

```

Decisión de almacenamiento — JSONB, no pgvector (para el diary). Conviene ser preciso acá: el embedding del diary se guarda como `JSONB` y la similitud se calcula **en memoria** con una `cosine` propia (`embed.ts:30`), no con el operador de pgvector ni con un índice ANN. El comentario del propio módulo lo justifica (`embed.ts:1-6`): a la escala del diary (cientos de lecciones) un cosine in-memory es *más simple, zero-dep y suficientemente rápido*; se eligió por encima de un índice vectorial pesado (TurboVec). pgvector **sí** está en la infra (Postgres 14 + pgvector, `_SHARED_FACTS.md`), pero lo usa el `rag-module` de las apps generadas, no el cerebro del swarm. El índice relevante del diary es por tanto el B-tree (`agent_id, created_at DESC`), que sirve la ruta de recencia y acota el `DISTINCT ON` de la ruta de relevancia (§ 6).

El embedding lo produce un endpoint OpenAI-compatible `/v1/embeddings` (`embed.ts:13`): por defecto `nomic-embed-text` local en Ollama (gratis, sin API key), o `text-embedding-3-small` con BYOK. Cualquier fallo devuelve `null` y los callers degradan a recencia.

Los mismos exemplars de código verificado (`verified_exemplars`) siguen el mismo patrón — `embedding JSONB`, índice por (`kind, created_at DESC`) (`genetic.sql:138-146`) — para recuperar código *known-good* por similitud al prompt (`exemplarFor`, `swarm-fitness.ts:44`, con piso de similitud `0.55`).

6. Retrieval Strategy (Similaridad · Ranking)

Hay dos rutas de recuperación, ambas reales:

Recencia (fallback simple). `recentLessons(agentId, n=4)` (`progression.ts:134`) y `teamLessonDigest(team, n=6)` (`progression.ts:141`) traen las últimas lecciones por `created_at DESC`. Es la ruta usada cuando no hay embeddings disponibles.

Relevancia semántica (ruta principal). `relevantLessons(team, taskText, n=6)` (`progression.ts:154`) es el corazón del retrieval. Embebe la tarea actual y rankea las lecciones del team por similitud coseno, con tres endurecimientos exigidos por las críticas internas:

```
const RELEVANCE_FLOOR = 0.35 // por debajo, lección de otro dominio → ignorar
const scored = rows.map((r) => {
  const v = /* parse del JSONB embedding */
  const sim = v.length ? cosine(q, v) : -1
  const ageDays = (now - new Date(r.created_at).getTime()) / 86400000
  const decay = Math.exp(-ageDays / 45) // half-life ~31 días
  return { entry: r.entry, sim, score: sim * (0.6 + 0.4 * decay) }
}).filter((s) => s.sim >= RELEVANCE_FLOOR).sort((a, b) => b.score - a.score)
if (!scored.length) return "" // nada relevante → ningún consejo gana a uno equivocado
return scored.slice(0, n).map((s) => `- ${s.entry}`).join("\n")
```

Los tres mecanismos verificados (`progression.ts:149-175`):

1. **Relevance floor 0.35**. Una lección de otro dominio (p.ej. fintech aplicada a salud) no se fuerza: si `sim < 0.35`, se descarta. **Devolver nada es válido**: "ningún consejo gana a un consejo equivocado" (`:173`).
2. **Recency decay**. El score final no es solo similitud: es `sim × (0.6 + 0.4·decay)` con `decay = e-(ageDays/45)` (half-life ~31 días). Una lección vieja y muy similar puede ceder ante una reciente y similar, evitando consejos contradictorios de versiones viejas del app. El peso de la similitud nunca baja de `0.6` (la mitad relevancia, la mitad frescura).
3. **Ranking + top-N**. Se ordena por `score` y se toman las `n` mejores (default 6), formateadas como bullets para el few-shot.

La query trae `DISTINCT ON (d.entry)` (dedup de lecciones idénticas compartidas por el team) con un tope de 400 filas candidatas (`progression.ts:159-162`), y **ya incorpora la compuerta anti-poisoning** descrita en el § 8 dentro del `WHERE`.

7. Memory Aging (Decay · Consolidación)

Decay (envejecimiento). No se borran lecciones viejas; se las **pondera hacia abajo**. El factor `decay = e-(ageDays/45)` del § 6 implementa un olvido suave con half-life de ~31 días: una lección de hace un mes vale la mitad (en su componente de frescura) que una recién escrita. Esto es *aging* sin pérdida — la lección sigue disponible si nada más reciente y relevante la supera.

Consolidación. Ocurre en dos planos, ambos en el código:

- *En el diary* — la deduplicación. Una lección que se repite muchas veces (porque el team la gana round tras round) **no inunda** el contexto: el `DISTINCT ON (d.entry)` la consolida a una sola fila al recuperar, y el `embCache` evita re-embeber el duplicado al escribir.
- *En los skills* — la consolidación "real" del conocimiento. Las lecciones son ruidosas y efímeras; la versión destilada y estable del conocimiento de un área vive en `puglit_skills`, donde **SkillOpt** consolida señal repetida en ediciones acotadas del *skill doc*, versionadas y validadas (§ 8). Es la diferencia entre memoria episódica (diary) y memoria procedural (skill).

El espejo humano de todo esto se sincroniza a un vault Obsidian (`syncObsidian`, `progression.ts:178`): por cada agente se escribe su ficha + sus últimas 20 entradas de diary (`progression.ts:182-184`), lo que da una vista auditable del aprendizaje.

8. Anti-Drift Mechanisms

El riesgo de un sistema que aprende de sí mismo es envenenar su propia memoria: que una build mala escriba una lección que después se recupera y degrada las builds siguientes. Puglit tiene dos compuertas independientes contra eso.

8.1 Anti-poisoning en el retrieval del diary

La ruta de relevancia **nunca recupera** lecciones de rounds malos o de builds que fallaron la compuerta de runtime. El filtro está en el `WHERE` de `relevantLessons` (`progression.ts:159-162`):

```
WHERE a.team = $1
AND d.embedding IS NOT NULL
AND COALESCE(d.quality, 60) >= 45          -- solo lecciones de rounds de calidad decente
AND COALESCE(d.outcome, 'unknown') <> 'failure' -- nunca una lección de build que falló el ga
```

- **quality ≥ 45** (sobre el `quality` 0-100-equivalente del round; recordar que `quality` en el diary se guardó como `score/10`, pero el umbral acá opera con `COALESCE(..., 60)` como default conservador para lecciones legacy sin score). Una build pobre produce lecciones de baja calidad, y esas no se reciclan.
- **outcome ≠ 'failure'** — la columna `outcome` (`genetic.sql:149`) la marca el gate de runtime; una lección de una build que no levantó o tiró 5xx queda excluida del recall, aunque su texto sea semánticamente atractivo.

Combinado con el **relevance floor 0.35** y el **decay**, el diary degrada *grácil* y *seguro*: loosely-related, low-quality, failed o stale → fuera.

8.2 SkillOpt: validation-gated skill training

`skill-evolution.ts` implementa el patrón **microsoft/SkillOpt**: el *skill doc* de cada área es el único estado entrenable de agentes congelados. Corre **offline** (estilo *SkillOpt-Sleep*), nunca inline por build, y su bucle es:

```
rollout → reflect/aggregate → propose (bounded) → VALIDATE (held-out) → select → update
```


Estado y seed. Hay cinco áreas de skill: `data | dev | design | business | test` (`skill-evolution.ts:19`). Cada una arranca de un playbook seed (`PLAYBOOK.*`, `:20`). En runtime, `skillFor(area)` devuelve el doc evolucionado **validado** si existe, si no el seed (`:27`); `loadActiveSkills()` lo carga una vez por build desde el `puglit_skills` activo más nuevo (`DISTINCT ON (area) ... ORDER BY area, version DESC`, `:35-37`).

Held-out validation set (frozen). La compuerta es un set de **5 tareas diversas y congeladas** (`VAL_TASKS`, `:42-48`): booking marketplace, used-goods con swipe/match, live football scores, team task manager, subscription box. Validar un skill = diseñar un blueprint compacto para cada tarea **usando ese skill** y promediar su `objectiveScore` (`validate`, `:67-70`; `rolloutScore`, `:60`). Como las tareas son fijas y diversas, **el skill tiene que generalizar, no overfittear un solo producto**. `objectiveScore` (`swarm-fitness.ts:17`) es determinístico (premia schemas reales, cobertura de rutas vs entidades, páginas; penaliza *phantom tables*), así que la compuerta no depende de un juez LLM ruidoso.

Bounded edits (el "learning rate" textual). El optimizer (un modelo `premium`) propone, sobre el doc actual, **a lo sumo 3 cambios** add/delete/replace, con **cambio neto ≤ 500 chars** y **doc final ≤ 1900 chars** (`:94`). Se le prohíbe **re-introducir ediciones ya rechazadas** y se le pasan las últimas 12 del buffer de rechazos (`rejectedEdits`, `:72`). Esto acota cuánto puede mutar el skill por época — el equivalente textual de una tasa de aprendizaje chica y estable. Un candidato fuera de rango (`<120` o `>2600` chars, o idéntico al actual) se descarta de entrada (`:99`).

Held-out gate (accept-if-strictly-better). El candidato se valida y **solo se acepta si supera al actual por un margen** (`:101-109`):

```
const after = await validate(candidate)
const MARGIN = Number(process.env.PUGLIT_SKILL_MARGIN || 1.5)
if (after > before + MARGIN) {
  // archivar el activo, insertar nueva version active, actualizar overlay
  await query("UPDATE puglit_skills SET status='archived' WHERE area=$1 AND status='active'", [a
  await query("INSERT INTO puglit_skills (area, version, doc, val_score, status) VALUES ($1,$2,$
  active[area] = { doc: candidate, score: after }
  return { accepted: true, before, after, edit }
}
```

El margen `1.5` (configurable vía `PUGLIT_SKILL_MARGIN`) evita *noise-chasing*: una mejora marginal indistinguible de ruido **no** se promueve. Cada aceptación **versiona** el skill (la versión vieja pasa a `archived`, nunca se pierde) — un historial auditable y reversible.

Rejected-edit buffer. Si el candidato no supera la compuerta, su edición se persiste en `puglit_skill_rejects` con sus scores before/after (`:110`):

```
await query("INSERT INTO puglit_skill_rejects (area, edit, before_score, after_score) VALUES ($1,
```

Ese buffer se vuelve a inyectar al optimizer en la próxima época (§ "bounded edits"), así que el sistema **no reintenta indefinidamente** una idea que ya demostró no funcionar. Es memoria de fracasos, complementaria a la memoria de éxitos del diary.

Esquema de soporte (`genetic.sql:124-135`):


```
CREATE TABLE IF NOT EXISTS puglit_skills (
  id BIGSERIAL PRIMARY KEY,
  area VARCHAR(16) NOT NULL, version INT NOT NULL, doc TEXT NOT NULL,
  val_score DOUBLE PRECISION DEFAULT 0, status VARCHAR(12) NOT NULL DEFAULT 'active',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_puglit_skills_area ON puglit_skills(area, version DESC);
CREATE TABLE IF NOT EXISTS puglit_skill_rejects (
  id BIGSERIAL PRIMARY KEY,
  area VARCHAR(16) NOT NULL, edit TEXT NOT NULL,
  before_score DOUBLE PRECISION, after_score DOUBLE PRECISION, created_at TIMESTAMPTZ DEFAULT NOW()
);
```

`evolveAllSkills()` (:115) corre una época sobre las cinco áreas en secuencia; cada `evolveSkill` falla cerrado (`{ accepted: false }`) si algo revienta, dejando el skill activo intacto.

El diagrama de la compuerta SkillOpt:

```
skill activo (o seed)
  |
  feedback(scorecard) + rejected buffer(12)
  |
  OPTIMIZER (premium): ≤3 edits, Δ≤500 chars, doc≤1900
  |
  candidato — fuera de rango? → reject (descarte temprano)
  |
  VALIDATE: 5 tareas frozen → objectiveScore promedio = after
  |
  after > before + 1.5 ?
  ├── sí → archive viejo · INSERT version active · overlay
  └── no → INSERT en puglit_skill_rejects (no se reintenta)
```

9. Evolution Metrics

La regla del módulo de métricas es explícita: "**medí por evidencia, no por mecanismo**" (`swarm-metrics.ts:1`). XP y niveles son el *mecanismo* observable; lo que valida la tesis son cuatro KPIs concretos, Postgres-backed (`swarm-metrics.ts:22`):

Métrica	Qué mide
<code>build_success</code>	first-build-success-rate (compiló a la primera)
<code>smoke_pass</code>	runtime smoke-pass-rate (levantó y sirvió páginas sin 5xx)
<code>judge_agreement</code>	acuerdo inter-juez en el panel del torneo
<code>ablation_swarm_win</code>	ablación swarm-vs-single (¿el swarm gana al agente solo?)

`recordMetric(name, value, meta)` inserta cada muestra en `puglit_metrics` (`swarm-metrics.ts:9`); `metricRate(name, sinceDays=30)` agrega el promedio sobre una ventana (:14); y `scorecard()` arma el dashboard de las cuatro (:22). Esquema y su índice (`genetic.sql:114–121`):

```
CREATE TABLE IF NOT EXISTS puglit_metrics (  
  id BIGSERIAL PRIMARY KEY,  
  name VARCHAR(48) NOT NULL,  
  value DOUBLE PRECISION NOT NULL,  
  meta JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);  
CREATE INDEX IF NOT EXISTS idx_metrics_name ON puglit_metrics(name, created_at DESC);
```

Hay un acople elegante: ese mismo `scorecard()` es el `feedback()` que alimenta al optimizer de SkillOpt (`skill-evolution.ts:77-82`). Las métricas de evidencia no solo *miden* la evolución — la **conducen**: el optimizer afila el skill doc hacia lo que demostrablemente sube esos rates.

Cierre

El Evolution Engine de Puglit es, en una frase: **modelos congelados + estado textual/estructurado que evoluciona bajo compuertas de evidencia**. La progresión RPG hace visible la dinámica; las lecciones embebidas del diary dan memoria episódica recuperable por relevancia, envejecida y anti-envenenada; y SkillOpt destila esa señal en memoria procedural — el *skill doc* — que solo cambia si gana, estrictamente y por margen, en un held-out frozen. Nada se aplica por haber sido generado; todo se aplica por haber **pasado una compuerta**.